

Databázy a vizualizácia

Normalizácia a denormalizácia

doc. Ing. Ladislav Körösi, PhD.

Ústav robotiky a kybernetiky
FEI STU Bratislava

Normalizácia

Prečo ju potrebujeme

Pri návrhu databázy môžeme vytvoriť jednu veľkú tabuľku, ktorá obsahuje všetky údaje.

Takýto návrh však spôsobuje problémy:

- duplicitné údaje (opakované informácie),
- problémy pri aktualizácii (nekonzistentné dáta),
- problémy pri vkladaní a mazaní (anomálie).

Príklad:

- opakujúci sa názov predmetu pri každom študentovi,
- zmena názvu → nutné upraviť viac riadkov.

Normalizácia tieto problémy odstraňuje rozdelením dát do viacerých tabuliek.

Normalizácia

Definícia

Normalizácia je proces efektívnej organizácie údajov v databáze.

Normalizácia

- eliminuje duplicitné údaje v relačných databázach (napríklad ukladanie rovnakých údajov vo viac ako jednej tabuľke),
- zaisťuje správnosť vzťahov medzi dátami - integritu dát (iba ukladanie súvisiacich údajov do tabuľky).

Obe ciele normalizácie sú dôležité lebo znižujú množstvo miesta, ktoré databáza spotrebúva, a zabezpečujú logické uloženie údajov.

Databázová komunita vytvorila rad usmernení na zabezpečenie normalizácie databáz. Tieto sa označujú ako normálne formy a sú očíslované od 1 (najnižšej formy normalizácie, označovanej ako prvá normálna forma alebo 1NF) až po 5 (piatu normálnu formu alebo 5NF). V praktických aplikáciách často používa normalizácia do 3NF.

Normalizácia databázy znamená rozdeliť databázu na maličké tabuľky tak, aby údaje neboli redundantné.

Bezstratová dekompozícia

Pod týmto pojmom si môžete predstaviť vašu schopnosť nestratiť žiadne údaje. Musíte zabezpečiť, aby nedošlo k tomu, že v prípade, že sa rozdelené údaje budete snažiť opätovne spojiť do jednej tabuľky, nestratíte pritom žiadne údaje.

Prvá normálna forma požaduje, aby

- tabuľka mala primárny kľúč,
- každý atribút obsahoval len atomické hodnoty, to znamená hodnoty ktoré sú ďalej nedeliteľné,
- tabuľka nemala opakujúce sa atribúty.

V uvedených príkladoch normalizácií modifikácie tabuliek budú hlavne s cieľom ukázať nesprávnu úpravu tabuliek.

1NF

Príklady - 1. príklad

Majme tabuľku Navsteвовane_kurzy

id_meno	kurz
123	100, 105, 102
200	105, 110, 999
222	111, 854, 100

Vyhovuje 1NF?

1NF

Príklady - 1. príklad

Majme tabuľku Navsteвовane_kurzy.

id_meno	kurz
123	100, 105, 102
200	105, 110, 999
222	111, 854, 100

Tabuľka nevyhovuje 1NF pretože atribút kurz nemá atomické hodnoty. Atomická hodnota by bola len napr. 100. Tu by nastal problém s vyhľadávaním a aktualizáciou neatomických údajov. Modifikujme tabuľku.

1NF

Príklady - 1. príklad

Majme tabuľku Navsteвовane_kurzy po úprave

id_meno	kurz1	kurz2	kurz3
123	100	105	102
200	105	110	999
222	111	854	100

Vyhovuje 1NF?

1NF

Príklady - 1. príklad

Majme tabuľku Navsteвовane_kurzy po úprave

id_meno	kurz1	kurz2	kurz3
123	100	105	102
200	105	110	999
222	111	854	100

Tabuľka nevyhovuje 1NF lebo atribút kurz (kurz1, kurz2 a kurz3) je opakujúci sa atribút. Bol by problém ak by niekto chcel absolvovať viac ako 3 kurzy. Jednalo by sa o zásah do štruktúry tabuľky, koľko max. kurzov?, rozšírenie všetkých existujúcich SQL príkazov, plytvanie miestom v DB (čo ak niekto absolvuje len 1 kurz), ... Modifikujme tabuľku.

1NF

Príklady - 1. príklad

Majme tabuľku Navsteвовane_kurzy po ďalšej úprave

id_meno	kurz
123	100
123	105
123	102
200	105
200	110
200	999

Vyhovuje 1NF?

Majme tabuľku Navsteвовane_kurzy po ďalšej úprave

id_meno	kurz
123	100
123	105
123	102
200	105
200	110
200	999

Tabuľka obsahuje atomické hodnoty a neopakujúce sa atribúty. Čo je PK? Stĺpce id_meno a kurz neobsahujú jedinečné záznamy. Ak by bol PK kombinácia oboch stĺpcov, nebolo by možné absolvovať ten istý kurz viac krát tým istým človekom. Modifikujme tabuľku.

1NF

Príklady - 1. príklad

Majme tabuľku Navsteвовane_kurzy ďalšej modifikácii.

id_zapisu (PK)	id_meno (CK)	kurz (CK)
1	123	100
2	123	105
3	123	102
4	200	105
5	200	110
6	200	999
7	123	100

Vyhovuje 1NF?

Majme tabuľku Navsteвовane_kurzy ďalšej modifikácii.

id_zapisu (PK)	id_meno (CK)	kurz (CK)
1	123	100
2	123	105
3	123	102
4	200	105
5	200	110
6	200	999
7	123	100

Tabuľka vyhovuje 1NF. Absolvent kurzu s `id_meno = 123` zapísal kurz = 100 až 2x (`id_zapisu = 1` a `id_zapisu = 7`). Stĺpec `id_meno` je CK a referuje na údaje v entite osoby. Stĺpec kurz je tiež CK a referuje na údaje v entite kurzy.

1NF

Príklady - 2. príklad

Majme tabuľku Osoba

meno	priezvisko	adresa	telefony
Jan	Novák	Havlíčková 2 Praha 3	123, 324, 3452
Petr	Kovář	Hlavná 15 Brno	345, 239, 284
Pavel	Pavel	Papalášova 25 Brno	888, 111, 333

Vyhovuje 1NF?

Majme tabuľku Osoba

meno	priezvisko	adresa	telefony
Jan	Novák	Havlíčková 2 Praha 3	123, 324, 3452
Petr	Kovář	Hlavná 15 Brno	345, 239, 284
Pavel	Pavel	Papalášova 25 Brno	888, 111, 333

Atribúty adresa a telefony obsahujú neatomické prvky. Tabuľka nemá PK. Tabuľka nevyhovuje 1NF. Modifikujme tabuľku.

1NF

Príklady - 2. príklad

Majme tabuľku Osoba po modifikácii

meno	priezvisko	adresa	tel1	tel2	tel3
Jan	Novák	Havlíčková 2 Praha 3	123	324	3452
Petr	Kovář	Hlavná 15 Brno	345	239	284
Pavel	Pavel	Papalášova 25 Brno	888	111	333

Vyhovuje 1NF?

Majme tabuľku Osoba po modifikácii

meno	priezvisko	adresa	tel1	tel2	tel3
Jan	Novák	Havlíčková 2 Praha 3	123	324	3452
Petr	Kovář	Hlavná 15 Brno	345	239	284
Pavel	Pavel	Papalášova 25 Brno	888	111	333

Atribút adresa obsahuje neatomické prvky. Tabuľka nemá PK. Telefón je opakujúcim sa atribútom. Tabuľka nevyhovuje 1NF. Modifikujme tabuľku.

1NF

Príklady - 2. príklad

Majme tabuľku Osoba po modifikácii

meno	priezvisko	ulica	c.	mesto	tel1	tel2	tel3
Jan	Novák	Havlíčková	2	Praha 3	123	324	3452
Petr	Kovář	Hlavná	15	Brno	345	239	284
Pavel	Pavel	Papalášova	25	Brno	888	111	333

Vyhovuje 1NF?

Majme tabuľku Osoba po modifikácii

meno	priezvisko	ulica	c.	mesto	tel1	tel2	tel3
Jan	Novák	Havlíčková	2	Praha 3	123	324	3452
Petr	Kovář	Hlavná	15	Brno	345	239	284
Pavel	Pavel	Papalášova	25	Brno	888	111	333

Tabuľka nemá PK. Telefón je opakujúcim sa atribútom. Tabuľka nevyhovuje 1NF. Modifikujme tabuľku.

1NF

Príklady - 2. príklad

Majme tabuľku Osoba po modifikácii

meno	priezvisko	ulica	c.	mesto	tel
Jan	Novák	Havlíčková	2	Praha 3	123
Jan	Novák	Havlíčková	2	Praha 3	324
Jan	Novák	Havlíčková	2	Praha 3	3452
Petr	Kovář	Hlavná	15	Brno	345
Petr	Kovář	Hlavná	15	Brno	239

Vyhovuje 1NF?

Majme tabuľku Osoba po modifikácii

meno	priezvisko	ulica	c.	mesto	tel
Jan	Novák	Havlíčková	2	Praha 3	123
Jan	Novák	Havlíčková	2	Praha 3	324
Jan	Novák	Havlíčková	2	Praha 3	3452
Petr	Kovář	Hlavná	15	Brno	345
Petr	Kovář	Hlavná	15	Brno	239

Tabuľka nemá PK. Tabuľka nevyhovuje 1NF. Modifikujme tabuľku.

1NF

Príklady - 2. príklad

Majme tabuľku Osoba po modifikácii

id	meno	priezvisko	ulica	c.	mesto	tel
1	Jan	Novák	Havlíčková	2	Praha 3	123
2	Jan	Novák	Havlíčková	2	Praha 3	324
3	Jan	Novák	Havlíčková	2	Praha 3	3452
4	Petr	Kovář	Hlavná	15	Brno	345
5	Petr	Kovář	Hlavná	15	Brno	239

Vyhovuje 1NF?

Majme tabuľku Osoba po modifikácii

id	meno	priezvisko	ulica	c.	mesto	tel
1	Jan	Novák	Havlíčková	2	Praha 3	123
2	Jan	Novák	Havlíčková	2	Praha 3	324
3	Jan	Novák	Havlíčková	2	Praha 3	3452
4	Petr	Kovář	Hlavná	15	Brno	345
5	Petr	Kovář	Hlavná	15	Brno	239

Tabuľka má PK, ale je nevhodne zvolený. Prvé 3 záznamy evidujú tú istú osobu, ale podľa id sú záznamy vedené ako rôzne osoby. Tabuľka nevyhovuje 1NF. Modifikujme tabuľku.

1NF

Príklady - 2. príklad

Majme tabuľky Osoba a Telefony

id (PK)	meno	priezvisko	ulica	c.	mesto
1	Jan	Novák	Havlíčková	2	Praha 3
5	Petr	Kovář	Hlavná	15	Brno

id (PK)	id_meno (CK)	tel.
1	1	123
2	1	324
3	1	3452
4	2	345
5	2	239
6	2	284

Vyhovujú 1NF?

1NF

Príklady - 2. príklad

Majme tabuľky Osoba a Telefony

id (PK)	meno	priezvisko	ulica	c.	mesto
1	Jan	Novák	Havlíčková	2	Praha 3
5	Petr	Kovář	Hlavná	15	Brno

id (PK)	id_meno (CK)	tel.
1	1	123
2	1	324
3	1	3452
4	2	345
5	2	239
6	2	284

Tabuľka Osoba obsahuje všetky údaje o osobe okrem telefónu. Telefóny sú uložené v novej tabuľke a priradené k osobám pomocou CK - stĺpca id_meno. Obe tabuľky vyhovujú 1NF.

1NF

Príklady - 3. príklad

Majme tabuľku Objednavky

zakaznik	produkt	cena
Thomas	Shirt	40
Maria	Tennis shoes	35
Evelyn	Shirt	40
Pajaro	Trousers	35

Vyhovuje 1NF?

Majme tabuľku Objednavky

zakaznik	produkt	cena
Thomas	Shirt	40
Maria	Tennis shoes	35
Evelyn	Shirt	40
Pajaro	Trousers	35

Tabuľka neobsahuje PK. Ak by sme zvolili za PK kombináciu stĺpcov zakaznik a produkt, nebolo by možné zákazníkom objednať druhý krát ten istý produkt. Otázkou je, či Tennis shoes budeme považovať za atomickú hodnotu, t. j. či je potrebné viesť záznam o produktoch ako shoes, shirts, ... a priradenie vlastností ako tennis, football, ... Pre tento príklad považujme hodnoty za atomické. Modifikujme tabuľku.

1NF

Príklady - 3. príklad

Majme tabuľku Objednavky po úprave

obj_c (PK)	zakaznik	produkt	cena
1	Thomas	Shirt	40
2	Maria	Tennis shoes	35
3	Evelyn	Shirt	40
4	Pajaro	Trousers	35

Vyhovuje 1NF?

Majme tabuľku Objednavky po úprave

obj_c (PK)	zakaznik	produkt	cena
1	Thomas	Shirt	40
2	Maria	Tennis shoes	35
3	Evelyn	Shirt	40
4	Pajaro	Trousers	35

Tabuľka vyhovuje 1NF. Pri vyšších NF zistíme, že tabuľka má obsahovať spolu súvisiace údaje. Produkt súvisí s cenou, ale nie so zákazníkom. Cena by súvisela s číslom objednávky, ak by sme cenu považovali za nákupnú a nie aktuálnu cenu produktu. Z údajov tabuľky je zrejmé, že existujú entity zákazník, produkt a mala by existovať prepojujúca tabuľka kvôli vzťahu M:N.

1NF

Príklady - 4. príklad

Majme tabuľku Objednavky

prod_id (PK)	farba	cena
1	červená, zelená	15.99
2	žltá	23.99
3	zelená	17.50
4	žltá, modrá	9.99

Vyhovuje 1NF?

1NF

Príklady - 4. príklad

Majme tabuľku Objednavky

prod_id (PK)	farba	cena
1	červená, zelená	15.99
2	žltá	23.99
3	zelená	17.50
4	žltá, modrá	9.99

Tabuľka obsahuje neatomické prvky. Má PK. Nemá opakujúce sa atribúty. Tabuľka nevyhovuje 1NF. Modifikujme tabuľku.

1NF

Príklady - 4. príklad

Majme tabuľky Ceny a Farby. Vyhovujú 1NF?

prod_id	cena
1	15.99
2	23.99
3	17.50
4	9.99

prod_id	cena
1	červená
1	zelená
2	žltá
3	zelená
4	žltá
4	modrá
5	červená

Majme tabuľky Ceny a Farby

prod_id	cena
1	15.99
2	23.99
3	17.50
4	9.99

prod_id	farba
1	červená
1	zelená
2	žltá
3	zelená
4	žltá
4	modrá
5	červená

Tabuľka Ceny vyhovuje 1NF. Čo je PK v tabuľke Farby? Za PK by sme mohli zvoliť kombináciu stĺpcov prod_id a farba. V tom prípade by tabuľka vyhovovala 1NF. Je to ale problematické lebo farby sa vyskytujú v tabuľke viac krát. Modifikujme tabuľku.

1NF

Príklady - 4. príklad

Majme nové tabuľky Ceny a Farby. Vyhovujú 1NF?

prod_id (PK)	cena
1	15.99
2	23.99
3	17.50
4	9.99

id (PK)	prod_id	cena
1	1	červená
2	1	zelená
3	2	žltá
4	3	zelená
5	4	žltá
6	4	modrá
7	5	červená

1NF

Príklady - 4. príklad

Majme nové tabuľky Ceny a Farby. Umelo sme pridali nový stĺpec id. Tabuľky vyhovujú 1NF.

prod_id (PK)	cena
1	15.99
2	23.99
3	17.50
4	9.99

id (PK)	prod_id	cena
1	1	červená
2	1	zelená
3	2	žltá
4	3	zelená
5	4	žltá
6	4	modrá
7	5	červená

Druhá normálna forma požaduje, aby

- tabuľka vyhovovala 1NF,
- a zároveň každý neklúčový atribút bol od primárneho kľúča úplne funkčne závislý (od celého kľúča, nie len od jeho podmnožiny).

Funkčná závislosť

Tento pojem sa snaží vyjadriť vzťah jednotlivých atribútov v danej entite, ktorý musí byť priamy voči primárnemu kľúču. Napríklad majme tabuľku HLAVIČKA_FAKTÚRY, ktorá bude udržiavať informácie ako číslo faktúry (Primárny kľúč), komu bude faktúra vystavená, kedy a kým.

Predstavme si, že by sme mali v tabuľke aj ďalší atribút pobočka zamestnanca. Informovala by, na pobočke zamestnanec vystaví faktúru. Tento atribút nie je funkčne závislý od čísla faktúry, lebo nepopisuje priamo faktúru. Tento atribút popisuje priamo pobočku, nie faktúru. Preto nie je funkčne závislý od faktúry. Preto tam nemá byť. Takže funkčná závislosť znamená, že každý atribút danej entity je priamo závislý na primárnom kľúči, ktorý sme si zvolili.

Funkčná závislosť sa zapisuje:

$$A \rightarrow B$$

Znamená:

- hodnota atribútu A jednoznačne určuje hodnotu atribútu B,
- pre každú hodnotu A existuje práve jedna hodnota B.

V tabuľke:

- ak majú dva riadky rovnakú hodnotu v stĺpci A,
- musia mať rovnakú hodnotu aj v stĺpci B.

Príklady:

- mesto $\rightarrow$ PSC (každé mesto má jedno PSČ)
- funkcia $\rightarrow$ plat (pre danú funkciu je určený plat)
- r.c. $\rightarrow$ meno (rodné číslo identifikuje konkrétnu osobu)

Poznámka: ide o vlastnosť dát v tabuľke, nie nutne vlastnosť reality.

2NF

Príklady - 1. príklad

Majme tabuľku Hodnotenie

student (PK)	kod_predmetu (PK)	nazov_predmetu	znamka
1	1	Základy automatizácie	B
1	2	Spojité procesy	A
2	2	Spojité procesy	B
3	1	Základy automatizácie	C

Vyhovuje 2NF?

2NF

Príklady - 1. príklad

Majme tabuľku Hodnotenie

student (PK)	kod_predmetu (PK)	nazov_predmetu	znamka
1	1	Základy automatizácie	B
1	2	Spojité procesy	A
2	2	Spojité procesy	B
3	1	Základy automatizácie	C

Tabuľka vyhovuje 1NF. Názov predmetu je závislý len od kódu predmetu a nie od študenta. Nakoľko nie je funkčne závislý od celého PK, nevyhovuje 2NF. Na prevod do 2NF by bolo potrebné tabuľku rozdeliť na dve, pričom z pôvodnej relácie by sme odstránili názov predmetu a vytvorili by sme novú reláciu. Vykonajme zmenu.

2NF

Príklady - 1. príklad

Majme tabuľky Hodnotenie a Predmety

student (PK)	kod_predmetu (PK)	znamka
1	1	B
1	2	A
2	2	B
3	1	C

kod_predmetu (PK)	nazov_predmetu
1	Základy automatizácie
2	Spojité procesy

Vyhovujú 2NF?

2NF

Príklady - 1. príklad

Majme tabuľky Hodnotenie a Predmety

student (PK)	kod_predmetu (PK)	znamka
1	1	B
1	2	A
2	2	B
3	1	C

kod_predmetu (PK)	nazov_predmetu
1	Základy automatizácie
2	Spojité procesy

Stĺpce student a kod_predmetu tvoria PK v tabuľke Hodnotenie. Zároveň sú aj CK pre spojenie údajov z iných tabuliek. Obe tabuľky vyhovujú 2NF. Tu by sme mohli uvažovať viacero termínov hodnotenia. V tom prípade kombinácia prvých dvoch stĺpcov už nemôže byť PK. Riešením by bolo pridanie nového stĺpca ID alebo dátumu skúšky či pod. Modifikujme tabuľku.

2NF

Príklady - 1. príklad

Majme tabuľky Hodnotenie a Predmety

ID (PK)	student (CK)	kod_predmetu (CK)	znamka
1	1	1	B
2	1	2	A
3	2	2	B
4	3	1	C

kod_predmetu (PK)	nazov_predmetu
1	Základy automatizácie
2	Spojité procesy

Vyhovujú 2NF?

2NF

Príklady - 1. príklad

Majme tabuľky Hodnotenie a Predmety

ID (PK)	student (CK)	kod_predmetu (CK)	znamka
1	1	1	B
2	1	2	A
3	2	2	B
4	3	1	C

kod_predmetu (PK)	nazov_predmetu
1	Základy automatizácie
2	Spojité procesy

Obe tabuľky vyhovujú 2NF.

2NF

Príklady - 2. príklad

Majme tabuľku Sklad

nazov (PK)	vyrobca (PK)	telefon	cena	mnozstvo
Mléčná čokoláda	Milka	+420123456789	30Kč	2500
Oříšková čokoláda	Milka	+420123456789	30Kč	2800
Tyčinka milkyway	Milka	+420123456789	10Kč	7000
Mléčná čokoláda	Orion	+420987654321	25Kč	5800
Oříšková horalka	Horalka	+420987654321	7Kč	4560

Vyhovuje 1NF a 2NF?

Majme tabuľku Sklad

nazov (PK)	vyrobca (PK)	telefon	cena	mnozstvo
Mléčná čokoláda	Milka	+420123456789	30Kč	2500
Oříšková čokoláda	Milka	+420123456789	30Kč	2800
Tyčinka milkyway	Milka	+420123456789	10Kč	7000
Mléčná čokoláda	Orion	+420987654321	25Kč	5800
Oříšková horalka	Horalka	+420987654321	7Kč	4560

Ak považujeme hodnoty v stĺpci nazov za atomické, tak tabuľka vyhovuje 1NF. Tabuľka neobsahuje opakujúce sa atribúty a kandidátom na PK je kombinácia stĺpcov nazov a vyrobca. Stĺpec telefon je závislý len od stĺpca vyrobca a nie od stĺpca nazov. Cena a množstvo závisia od celého PK. Tabuľka nevyhovuje 2NF. Upravme tabuľku.

Anomálie: Ak by sa vymazali výrobky Milka, tak by sa stratilo tel. číslo výrobcu.

2NF

Príklady - 2. príklad

Majme tabuľky Sklad a Vyrobcu

nazov (PK)	vyrobca_id (PK)	cena	mnozstvo
Mléčná čokoláda	1	30Kč	2500
Oříšková čokoláda	1	30Kč	2800
Tyčinka milkyway	1	10Kč	7000
Mléčná čokoláda	2	25Kč	5800
Oříšková horalka	3	7Kč	4560

vyrobca_id (PK)	vyrobca	tel
1	Milka	+420123456789
2	Orion	+420987654321
3	Horalka	+420897654321

Vyhovujú 2NF?

Majme tabuľky Sklad a Vyrobcu

nazov (PK)	vyrobca_id (PK)	cena	mnozstvo
Mléčná čokoláda	1	30Kč	2500
Oříšková čokoláda	1	30Kč	2800
Tyčinka milkyway	1	10Kč	7000
Mléčná čokoláda	2	25Kč	5800
Oříšková horalka	3	7Kč	4560

vyrobca_id (PK)	vyrobca	tel
1	Milka	+420123456789
2	Orion	+420987654321
3	Horalka	+420897654321

Stĺpec vyrobca_id v tabuľke Sklad je zároveň aj CK pre tabuľku Vyrobcu. Takto vytvorené tabuľky vyhovujú 2NF.

Tretia normálna forma požaduje, aby

- tabuľka vyhovovala 2NF,
- a žiaden neklúčový atribút nie je tranzitívne funkčne závislý od primárneho kľúča. To znamená, že všetky neklúčové atribúty musia byť navzájom nezávislé.

3NF

Príklady - 1. príklad

Majme tabuľku

id_predmetu (PK)	nazov_predmetu	id_ucitela	meno_ucitela
1	Spojité procesy	100	Ucitel'1
2	Databázy a vizualizácia	101	Ucitel'2
3	Robotika	102	Ucitel'3

Vyhovuje 3NF?

3NF

Príklady - 1. príklad

Majme tabuľku

id_predmetu (PK)	nazov_predmetu	id_ucitela	meno_ucitela
1	Spojité procesy	100	Ucitel'1
2	Databázy a vizualizácia	101	Ucitel'2
3	Robotika	102	Ucitel'3

Existuje tranzitívna závislosť $\text{id_predmetu} \rightarrow \text{id_ucitela} \rightarrow \text{meno_ucitela}$, preto tabuľka nevyhovuje 3NF. Riešením je vytvorenie novej entity učiteľ a odstránením mena učiteľa. Upravme tabuľku.

3NF

Príklady - 1. príklad

Majme tabuľky Predmety a Ucitelia

id_predmetu (PK)	nazov_predmetu	id_ucitela (CK)
1	Spojité procesy	100
2	Databázy a vizualizácia	101
3	Robotika	102

id_ucitela (PK)	meno_ucitela
100	Učiteľ1
101	Učiteľ2
102	Učiteľ3

Vyhovujú 3NF?

3NF

Príklady - 1. príklad

Majme tabuľky Predmety a Ucitelia

id_predmetu (PK)	nazov_predmetu	id_ucitela (CK)
1	Spojité procesy	100
2	Databázy a vizualizácia	101
3	Robotika	102

id_ucitela (PK)	meno_ucitela
100	Učiteľ1
101	Učiteľ2
102	Učiteľ3

Tabuľky vyhovujú 3NF.

Majme tabuľku

r.c.	meno	priezvisko	mesto	PSC	funkcia	plat
1	Jack	Smith	Jihlava	58601	CEO	150000
2	Franta	Vomáčka	Praha10	10000	Senior Software Architect	80000
3	Pepa	František	Plzeň	12345	Senior Software Architect	80000
4	Pavel	Novák	Kocourkov	99999	Junior Developer	30000
5	Petr	Koukal	Praha10	10000	Database Designer	75000
6	Honza	Novák	Plzeň	12345	Junior Developer	30000

Funkčné závislosti:

- r.c. → mesto, mesto → PSC
- r.c. → funkcia, funkcia → plat

Problém: existujú tranzitívne závislosti (PSC a plat nezávisia priamo od r.c.).

Riešenie: rozdeliť tabuľku na *zamestnanec*, *mesto*, *funkcia*.

3NF

Príklady - 2. príklad

Majme tabuľky Mesto, Funkcia a Zamestnanec, ktoré vyhovujú 3NF.

mesto_id	mesto	PSC
1	Jihlava	58601
2	Praha10	10000
3	Plzeň	12345
4	Kocourkov	99999

funkcia_id	funkcia	plat
1	CEO	150000
2	Senior Software Architect	80000
3	Database Designer	75000
4	Junior Developer	30000

r.c.	meno	priezvisko	mesto_id	funkcia_id
1	Jack	Smith	1	1
2	Franta	Vomáčka	2	2
3	Pepa	František	4	2
4	Pavel	Novák	3	4
5	Petr	Koukal	2	3
6	Honza	Novák	4	4

3NF

Príklady - 3. príklad

Majme tabuľku:

- student_id, studijny_program, fakulta

Funkčné závislosti:

- student_id $\rightarrow$ studijny_program
- studijny_program $\rightarrow$ fakulta

Fakulta nezávisí priamo od student_id $\Rightarrow$ porušenie 3NF.

Riešenie:

- Student(student_id, studijny_program)
- Program(studijny_program, fakulta)

Návrh databázy prebieha v niekoľkých krokoch:

- návrh entít a vzťahov (ERA model),
- prevod na relačný model (tabuľky),
- úprava pomocou normalizácie.

Normalizácia:

- odstraňuje redundanciu,
- zabezpečuje konzistentnosť dát,
- zjednodušuje prácu s databázou.

Výsledkom je návrh, ktorý je vhodný na implementáciu v SQL databáze.

Normalizácia

Prečo nestačí ERA diagram

ER diagram predstavuje konceptuálny návrh databázy.

Po jeho prevode na tabuľky môžu vzniknúť problémy:

- nevhodne zvolené atribúty v tabuľke,
- redundantné údaje,
- nesprávne závislosti medzi atribútmi.

ER diagram nerieši:

- funkčné závislosti,
- redundanciu na úrovni tabuliek.

Preto je potrebná normalizácia, ktorá upravuje tabuľky do správnej formy.

Denormalizácia je opačný proces k normalizácii.

Znamená:

- zámerné pridanie redundancie do databázy,
- spojenie údajov z viacerých tabuliek.

Cieľ:

- zrýchliť čítanie dát (SELECT),
- znížiť počet JOIN operácií.

Denormalizácia

Prečo ju používame

Normalizácia zlepšuje štruktúru dát, ale:

- vedie k viacerým tabuľkám,
- dopyty vyžadujú JOIN operácie,
- môže znížiť výkon pri veľkých dátach.

Denormalizácia sa používa:

- pri veľkých databázach,
- pri častom čítaní dát,
- keď je výkon dôležitejší než minimalizácia redundancie.

Denormalizácia prináša aj problémy:

- duplicita dát,
- zložitejšie INSERT a UPDATE operácie,
- riziko nekonzistencie dát.

Preto:

- používa sa len v odôvodnených prípadoch,
- najprv sa databáza navrhuje normalizovane.